

Orbit Wars — v12 MCTS

An AlphaZero-style PUCT search over the bit-exact forward model: net prior on sampled candidate moves, bounded leaf value, snapshot/restore expansion, and a step-scheduled deploy time budget — a proof of concept, *not* wired into training

Source of truth: `orbit_wars_v12/mcts.py`

2026-06-19

Abstract

This is document 4 of four. It documents `mcts.py`: a Monte-Carlo Tree Search that uses the document 1 network as a *prior over sampled candidate actions* and a bounded *leaf value*, so a (small) policy can spend inference compute to play better and so search visit distributions can later serve as policy-improvement targets. It reuses document 1’s `GpuEnv`, observation encoding, and gated-allocation action verbatim; it does *not* use the reward (document 1) or the league — it scores leaves with a heuristic and plays a fixed scripted opponent. **This is an explicit proof of concept:** the search plumbing is verified, but it is over the deploy time budget and a small searched net still loses the head-to-head; the intended payoff is a BC-warm-started self-play distillation loop, which is future work. Status is in Section 7.

Contents

1	Scope and simplifications	1
2	Snapshot and restore	2
3	Node, candidates, and the prior	2
4	Selection, expansion, backup	2
5	Deploy agent (<code>MctsAgent</code>)	3
6	Evaluation harness	3
7	Status (proof of concept)	3

1 Scope and simplifications

The v12 stack already provides everything AlphaZero needs except the search: a bit-exact, fast forward model (`GpuEnv`) and a net with a prior + value head. `mcts.py` adds the search, under three deliberate simplifications:

- **Fixed deterministic opponent** (`opponent_action` codes). With the opponent fixed, the game is a deterministic *single-agent* MDP from the ego’s view, so plain PUCT is sound —

the simultaneous-move subtlety is dodged on purpose. The real self-play fix (decoupled-UCT / regret matching per node, 2p zero-sum first) is deferred until there is a BC-warm-started net to self-play.

- **Single env** ($B = 1$). A node stores a full state snapshot; expansion restores it into one shared work env and steps once. Batched-across-games search is a later optimization.
- **Bounded heuristic leaf value** by default (`value="net"` uses the value head instead). The PPO critic is policy-biased and historically lost to exactly this heuristic for leaf eval; an AlphaZero loop is what would later de-bias the head.

Determinism (required for correct snapshot/restore and reproducible search) holds with official comets (waypoint playback, no `torch.rand`) and a deterministic scripted opponent (codes 1/3/4/5; avoid 0=random). Candidate ego actions are sampled once and stored on the node, so the transitions never re-sample.

2 Snapshot and restore

A node owns a snapshot: `snapshot_env` clones the mutable `GpuEnv` state (the planet/fleet tensors, `ang_vel`, `step_ct`, `done`, and the comet `c_slot`) into a plain dict; `restore_env` writes it back, *re-cloning on the way in* so the in-place `env_step` writes of a simulation never corrupt the stored snapshot. The static official-comet arrays (`c_paths/c_len/c_ships`) are read-only during stepping and are shared by reference (bound once on the work env), so only `c_slot` is cloned.

3 Node, candidates, and the prior

A `_Node` stores its state snapshot plus K candidate edges with prior P , visit counts N , and value sums W (all on CPU — the per-node bookkeeping is tiny and per-sim `.item()` syncs would otherwise dominate).

Candidates (one net forward). `_candidates` builds the K candidate full-turn actions as the policy-greedy move (index 0, a safe fallback) plus $K-1$ samples. Crucially, all K share an *identical* observation, so the policy distribution is built *once* (one trunk+heads forward via `_make_dist`) and sampled K times; the prior is

$$P = \text{softmax}(\log \pi(a_1), \dots, \log \pi(a_K)),$$

the softmax of the candidates' own policy log-probs. (Re-running `act` per candidate — the naive version — would cost $K \times$ the entire search budget for a distributionally identical result; this dedup is what makes it a real search instead of a depth-1 stub.)

4 Selection, expansion, backup

PUCT selection. At a node, with total visits $N_{\text{tot}} = \sum_a N_a$,

$$a^* = \arg \max_a \left[Q_a + c_{\text{puct}} P_a \frac{\sqrt{N_{\text{tot}}}}{1 + N_a} \right], \quad Q_a = \begin{cases} W_a/N_a & N_a > 0 \\ 0.5 & \text{(first-play urgency)} \end{cases}$$

with $c_{\text{puct}} = \text{c_puct}$ (default 1.5). Unvisited edges get a neutral FPU of 0.5.

Expansion (`_expand_child`). Restore the node’s snapshot into the work env, apply the chosen edge — ego candidate + scripted opponent — with one `env_step`, and `settle`. The child is *terminal* if either side died or the horizon is reached, with terminal value

$$v = \begin{cases} 1 & \text{ego alive, opponent dead} \\ 0 & \text{ego dead, opponent alive} \\ 0.5 & \text{both dead} \\ \mathbf{1}[s_0 > s_1] + \frac{1}{2}\mathbf{1}[s_0 = s_1] & \text{horizon, by strength} \end{cases}$$

otherwise it is a fresh leaf whose value is the bounded leaf estimate and which stores its own snapshot + candidates.

Leaf value. `value="heuristic"` (default): a logistic of the ego’s strength margin against all opponents pooled,

$$v = \sigma\left(\frac{1}{\text{VALUE_SCALE}}[(\text{ships} + \text{fleet} + \text{PROD_WEIGHT} \cdot \text{prod})_{\text{ego}} - (\cdots)_{\text{opp}}]\right) \in [0, 1],$$

with `PROD_WEIGHT = 20` (1 production $\approx$ 20 ships of long-run value) and `VALUE_SCALE = 200`; this matches `settle_n`’s accounting. `value="net"` instead reads the value head, $\sigma(V/\text{VALUE_SCALE})$.

Backup. The leaf value is propagated up the visited path: $N_a += 1$, $W_a += v$ for each (node, a) on the path. (Single-agent: the opponent is baked into the transition, so the value is not negated between plies.)

Search loop (`mcts_search`). Up to `n_sims` PUCT simulations from the root, descending by selection until an unexpanded edge (`expand`) or a terminal child, then backing up. A `time_budget` (seconds) stops early — the most-visited edge so far is always a safe answer (greedy is edge 0). It returns the most-visited root action plus stats (`visits`, `sims_run`, `best_idx`, `switched = search preferred a non-greedy move`, `root_q`, `visit_entropy`, `elapsed`).

5 Deploy agent (`MctsAgent`)

`MctsAgent` plays seat 0 with the most-visited move under a **step-scheduled wall-clock budget**. `schedule` is a tuple of (`until_step`, `phase_seconds`); the per-turn budget inside a phase is `phase_seconds/phase_length`. The default

$$\text{DEFAULT_SCHEDULE} = ((20, 30.0), (60, 20.0), (80, 5.0))$$

spends 1.5/0.5/0.25 s per turn over steps 0–20 / 20–60 / 60–80, i.e. the per-game compute bank is front-loaded into the opening (where the game is decided) and tapers to greedy: $30 + 20 + 5 = 55$ s of a `bank_s = 60` s bank, leaving a `reserve_s = 5` s *untouchable* turbulence reserve that absorbs latency spikes so one slow turn cannot blow the game’s total time. Past the last phase, or once bank – reserve is spent, it plays **pure greedy** (one forward, no search). It reuses a single work env and binds the static comet arrays on first use. (`criticality()` — a cheap “how close is the game” measure in $[0, 1]$ — is available for a criticality-weighted budget, but the shipped `MctsAgent` budget is purely step-scheduled; the `crit_*` constructor args are deprecated and ignored.)

6 Evaluation harness

`play_2p_vs_script` plays seat 0 (driven by any `act_fn(env,t)->action`) against a scripted seat-1 bot on one world and returns the seat-0 score (1 win / 0.5 draw / 0). `greedy_act_fn` wraps the raw policy-greedy move (no search) as such an `act_fn`, so the same harness measures the search lift over greedy on identical worlds. The dedicated script `scripts/eval_mcts_192x6.py` runs a small searched net with breadth-100 search against the production trunks.

7 Status (proof of concept)

- **Plumbing verified.** `scripts/_mcts_smoke.py` exercises snapshot/restore, candidate dedup, PUCT, and the budget schedule (PASS).
- **Over the deploy budget.** On CPU the search runs ~ 1.9 s/turn — above the per-turn budget the schedule can afford — so at deploy it falls back to greedy for much of the game. Batched / GPU-resident search is the obvious next optimization.
- **A small searched net still loses 2p H2H.** The 192×6 MCTS net *with* breadth-100 search loses seat-0 2p head-to-head to the three production trunks (256×32 / 512×32 / 512×86); mean ≈ 0.167 . The search lift over greedy is only $\approx +0.056$ on average ($+0.167$ vs the weakest opponent, ≈ 0 vs the strong ones). This is a seat-0/2p-only PoC; the greedy opponent-model mismatch makes single games high-variance, and 4p is not runnable as a single-agent MDP. See memory `mcts-192x6-eval`.
- **The intended payoff is the loop, not this file.** The plan is BC warm-start *then* self-play distillation (search visit distributions as policy targets); only then does the simultaneous-move treatment (decoupled-UCT / regret matching, 2p zero-sum first) become necessary. DUCT is deferred until there is a warm-started net to self-play. See memory `mcts-poc-v12`.

Config. MCTS is not driven by `Config` fields — its knobs are `MctsAgent` / `mcts_search` arguments: `K` (candidates/node, default 6), `c_puct` (1.5), `value` ("heuristic" / "net"), `n_sims` / `max_sims` (sim cap), and the `schedule` / `bank_s` / `reserve_s` time budget. It consumes a document 1 checkpoint as its prior+value net.